ASTANA IT UNIVERSITY

Faculty of Computer Science & Engineering • Department of Computer Engineering

LABORATORY REPORT
Assignment #5: JavaScript Basics (DOM Manipulation & Events)

Course:	Web Technologies (Front-End Development)
Student Name:	**Aitpayev Tamerlan**
Academic Group:	**IT-2510**
Instructor:	Abitova G.A.
GitHub Repository:	https://github.com/tamerlanaitpayev1-netizen/web5

Selected Task Options:

- **Task 1: Option B — Turn-based Battle Game** (Hero vs Enemy with 100 HP, attack, shield defense, healing potions, HP percentage bars, last 5 actions log).
- **Task 2: Option A — Agile Kanban Board** (12 task objects, strict 3-card WIP limit, manual loop metrics, event delegation, zero innerHTML).
- **Task 3: Option A — Ticket Pass Builder** (7 fields, live boarding pass preview, 4 custom non-HTML rules, native Constraint Validation API).

1. Aim and Objectives of the Work

The objective of this laboratory assignment is to develop robust client-side interactive modules using standard ES6+ JavaScript, Document Object Model (DOM) programming, and event listeners. The implementation emphasizes defensive coding, responsive layouts, accessibility (WCAG AAA), and strict modular separation without third-party frameworks.

2. Task 1: Turn-Based Combat Game (Option B)

Task 1 models a stateful tactical combat simulator between the Valiant Hero and the Shadow Overlord. Both fighters have 100 HP. The player executes actions via distinct buttons:

- **Strike:** Deals 8–15 physical damage using `Math.floor(Math.random() * (15 - 8 + 1)) + 8` with a 10% chance of critical hit (2.0x damage).
- **Defend:** Halves next incoming enemy hit (50% damage reduction).
- **Heal (+20 HP):** Restores health, limited to 3 charges. Attempts when exhausted or at full HP are safely rejected without advancing the turn.

Logical Conditions (&&, ||, ===):

- *Draw:* `if (currentHeroHp <= 0 && currentEnemyHp <= 0)`
- *Victory:* `else if (currentEnemyHp <= 0 && currentHeroHp > 0)`
- *Defeat:* `else if (currentHeroHp <= 0 || (currentEnemyHp > 0 && currentHeroHp === 0))`

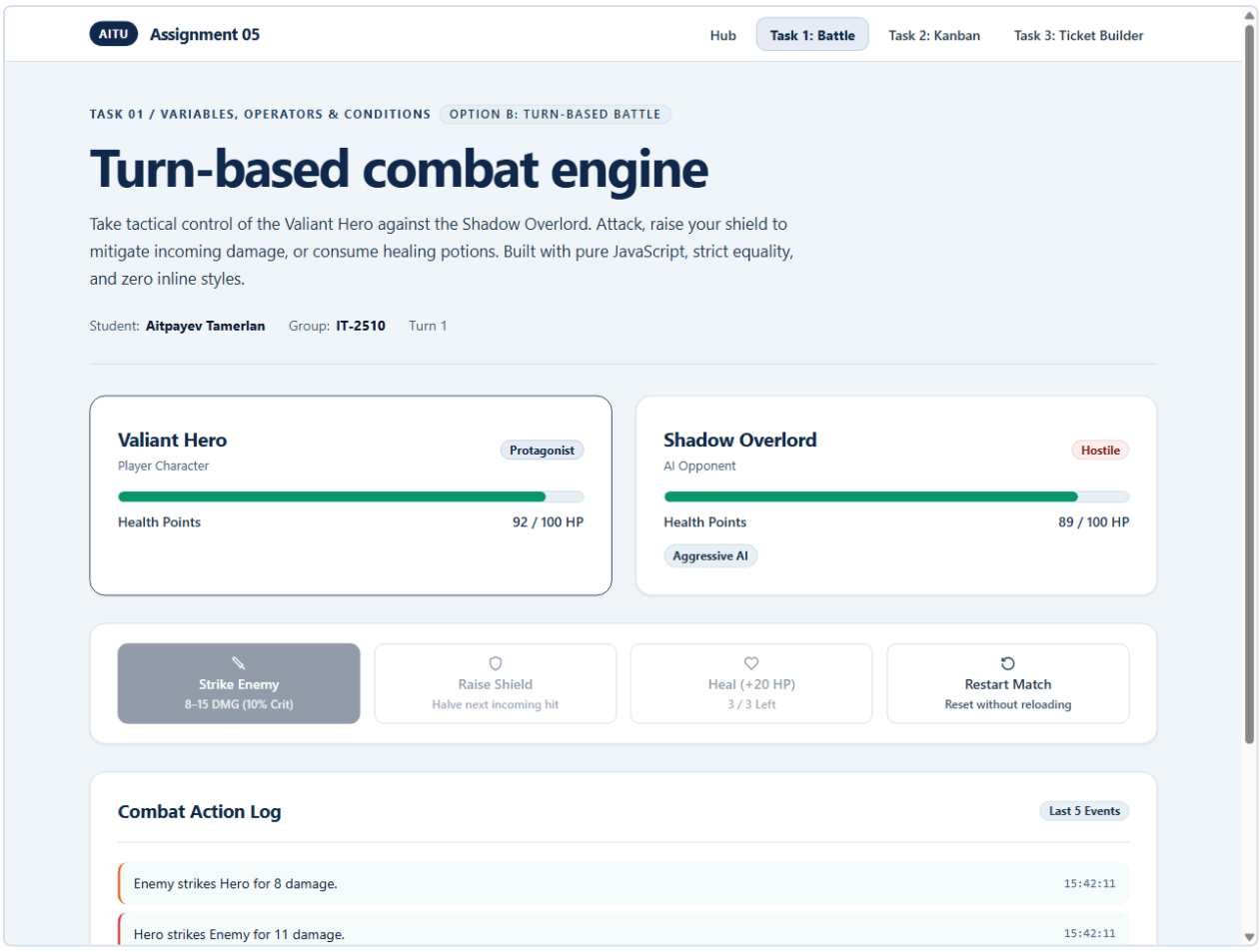


Figure 2.1: Turn-based battle combat interface with HP percentage bars, shield buff indicator, and chronological action log.

3. Task 2: Data-Driven Kanban Board (Option A)

Task 2 presents an agile project board constructed from an in-memory array of 12 rich task objects. The HTML contains only empty containers (`#todo-cards-container`, `#progress-cards-container`, `#done-cards-container`).

- **Zero innerHTML for Data:** Cards, badges, and controls are created strictly via `document.createElement`, `textContent`, `classList`, and `dataset`.
- **WIP Limit Rule:** The 'In progress' column holds at most 3 cards (`WIP_LIMIT = 3`). Moving a fourth card is blocked with an explicit warning banner.
- **Manual Loop Metrics:** A `for...of` loop calculates total cards, completed cards, velocity story points, and completion percentage.
- **Event Delegation:** A single container listener on `#kanban-board-container` catches clicks via `event.target.closest('[data-action]')`.

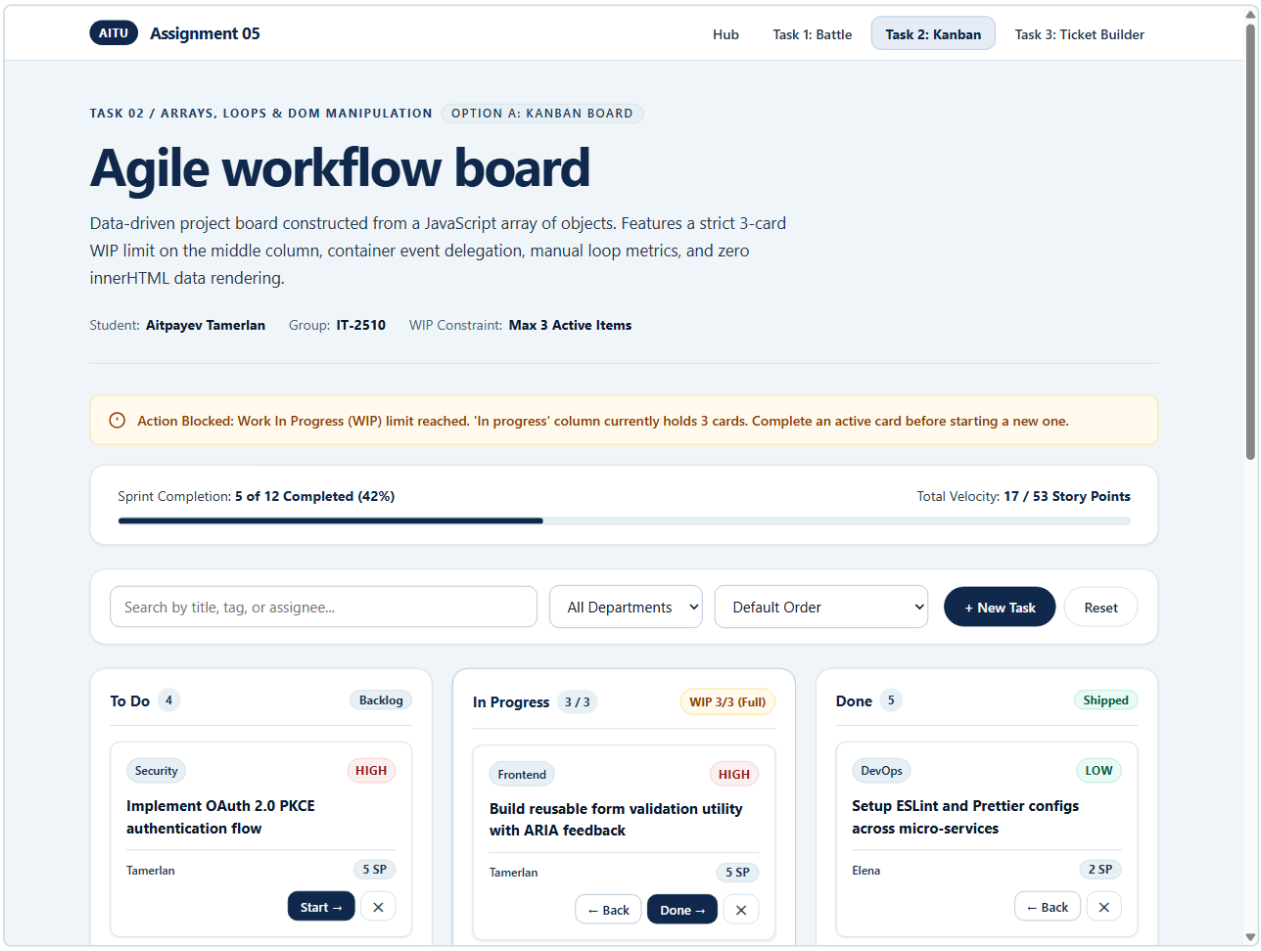


Figure 3.1: Agile Kanban board with WIP limit enforcement (blocking move at 3 cards) and live search/filter toolbar.

4. Task 3: Live Pass Builder & Form Validation (Option A)

Task 3 implements an event ticketing pass builder with live boarding pass preview synchronization. Native browser tooltips are disabled with `form.noValidate = true`. Fields are checked on blur, then live on input,

updating `aria-invalid` and error messages. On submit with errors, error count is announced and focus moves to the first invalid input.

Table 4.1: Specification of Validation Rules (Task 3 Matrix)

Field	Rule Description	Valid Example	Invalid Example & Reason
Tier & Quantity	Dynamic seat inventory: Quantity cannot exceed remaining seats in active tier object.	VIP Pass with `Quantity = 3` (4 available)	VIP Pass with `Quantity = 5` (Exceeds 4 seats in inventory)
Event Date	Sold-out & buffer restriction: Dates in blackout array (2026-11-15, 2026-12-25) or < 2 days in advance are rejected.	`2026-11-20` (Active conference day)	`2026-11-15` (Sold out gala) or past date
Promo Code	Catalog verification: Optional code must exist in active promotional registry to grant discount.	`AITU20` (20% off), `STUDENT50` (50% off)	`SALE99` (Unrecognized promo code)
Full Name	Multi-token legal name structure: Must contain at least two words (First + Last), min 2 chars per token, letters only.	`Tamerlan Aitpayev`, `Айтпаев Тамерлан`	`Tamerlan` (Single word) or `Tamerlan123`

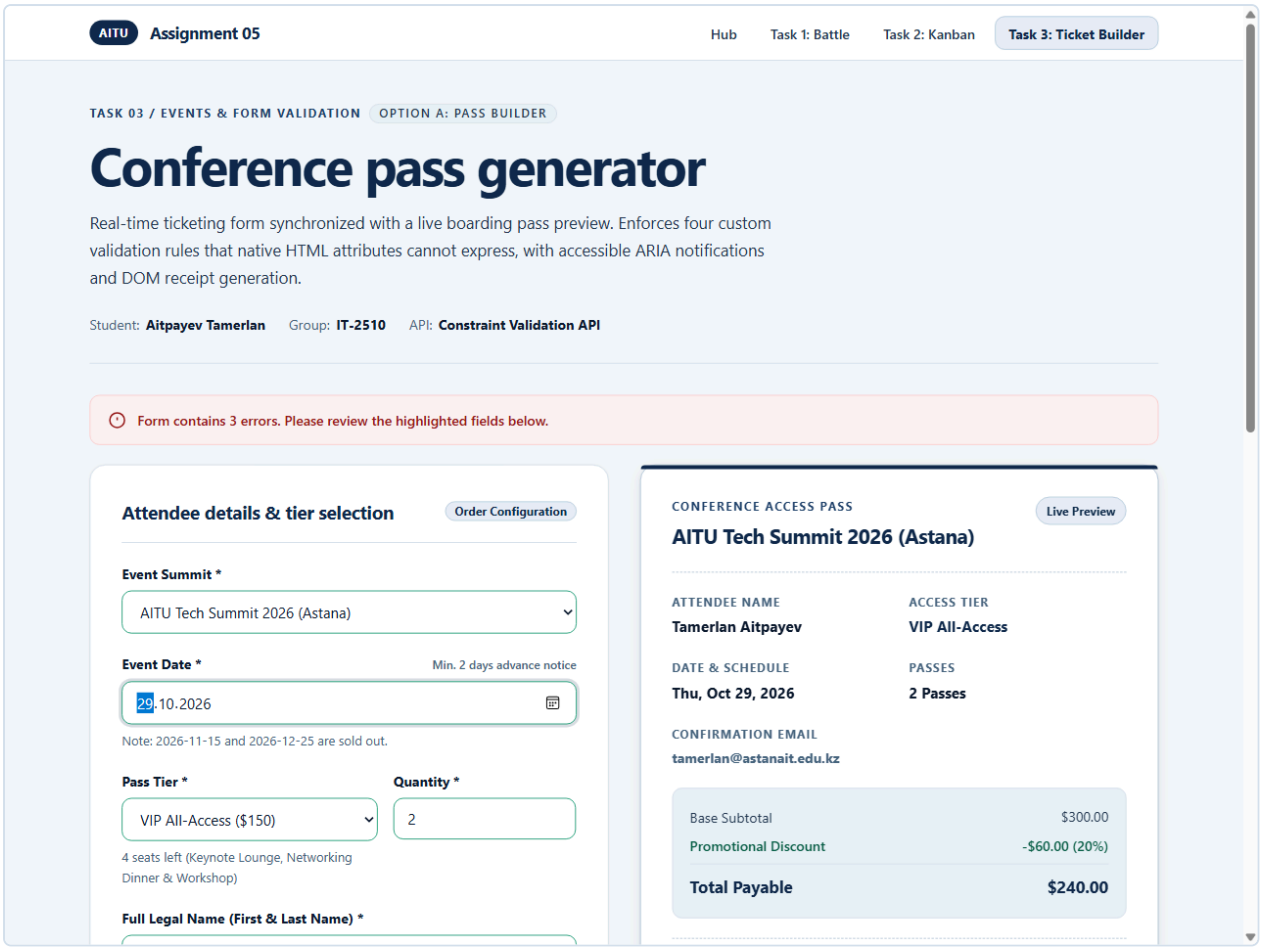


Figure 4.1: Pass builder form with real-time boarding pass synchronization and promo code discount.

5. Algorithmic Complexity & Architecture (Big-O Analysis)

- **Task 1 (Combat Engine):** All combat state mutations execute in `O(1)` time. Log maintenance with bounded length (`MAX_LOG_ENTRIES = 5`) runs in `O(1)` time and `O(1)` space.
- **Task 2 (Kanban Board):** Metrics loop traverses `N` tasks in `O(N)` time. Priority sorting operates in `O(N log N)` via TimSort. DOM tree generation takes `O(N)` time. Container event delegation operates in `O(1)` dispatch time.
- **Task 3 (Form Validation):** Validation runs across `K = 7` fields in `O(1)` time. Inventory lookup via `Array.find()` over `M = 3` tiers takes `O(1)` time.

6. Control Questions and Answers

Q1: What is the primary difference between == and === in JavaScript?

A1: `==` performs implicit type conversion before comparing values (e.g. `'0' == 0` evaluates to true). `===` performs strict equality checking without type coercion, requiring both operands to share identical data types and values.

Q2: Why is container-level Event Delegation preferred over attaching listeners to individual child elements?

A2: Event delegation utilizes DOM event bubbling. Attaching a single listener to the parent container conserves memory and automatically handles dynamic elements (e.g. newly created or moved cards) without re-binding listeners.

Q3: Why was innerHTML prohibited for data rendering in Task 2?

A3: Assigning strings to `innerHTML` introduces Cross-Site Scripting (XSS) risks and triggers expensive HTML parser tear-down. Utilizing `document.createElement` and `textContent` ensures complete sanitization and optimal rendering performance.

Q4: How does form.noValidate = true interact with the Constraint Validation API?

A4: Setting `form.noValidate = true` silences default browser tooltip popups, while preserving full access to the programmatic API methods (`checkValidity()`, `setCustomValidity()`).

7. Conclusion

During this assignment, I implemented three modern web modules adhering to strict front-end standards: a turn-based tactical combat simulator with fair random distributions, a data-driven Kanban board enforcing WIP constraints and event delegation with zero innerHTML, and an accessible pass builder utilizing the Constraint Validation API. The project features an editorial Naval Blue and crisp white design system verified at 375px and 1280px viewports with zero console errors.

8. References

[1] Abitova G.A. *Web technologies Front-End Development. Part 2.* Astana IT University, 2022.
[2] MDN Web Docs. *Constraint Validation API & DOM Manipulation.* Mozilla Corporation, 2026.

[3] ECMA International. *ECMAScript 2026 Language Specification (ECMA-262)*.
[4] W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*.